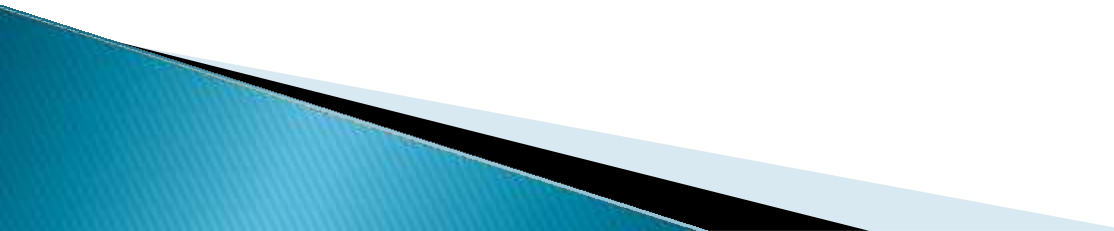


Introduction to Stacks

Module 2

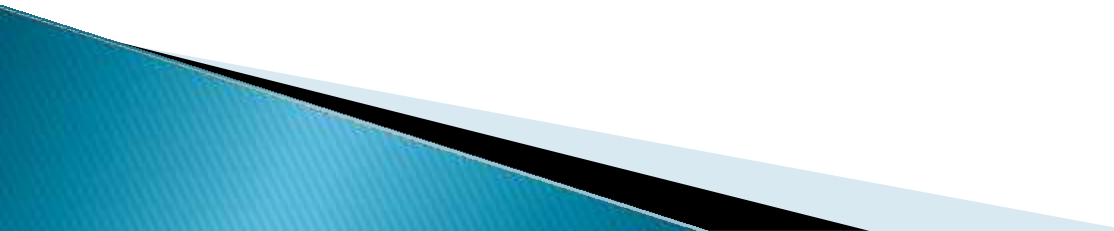
Dr. S. M. Farooq
Assistant Professor Senior
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore



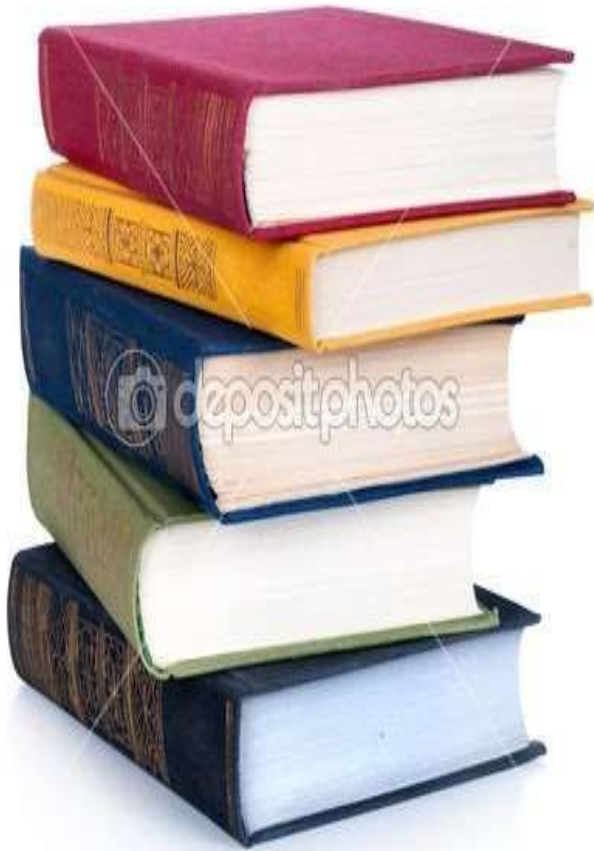
Introduction to Stack

A stack is called a last-in-first-out (LIFO) collection. This means that the last thing we added (pushed) is the first thing that gets pulled (popped) off.

A stack is called a last-in-first-out (LIFO) collection. This means that the last thing we added (pushed) is the first thing that gets pulled (popped) off.

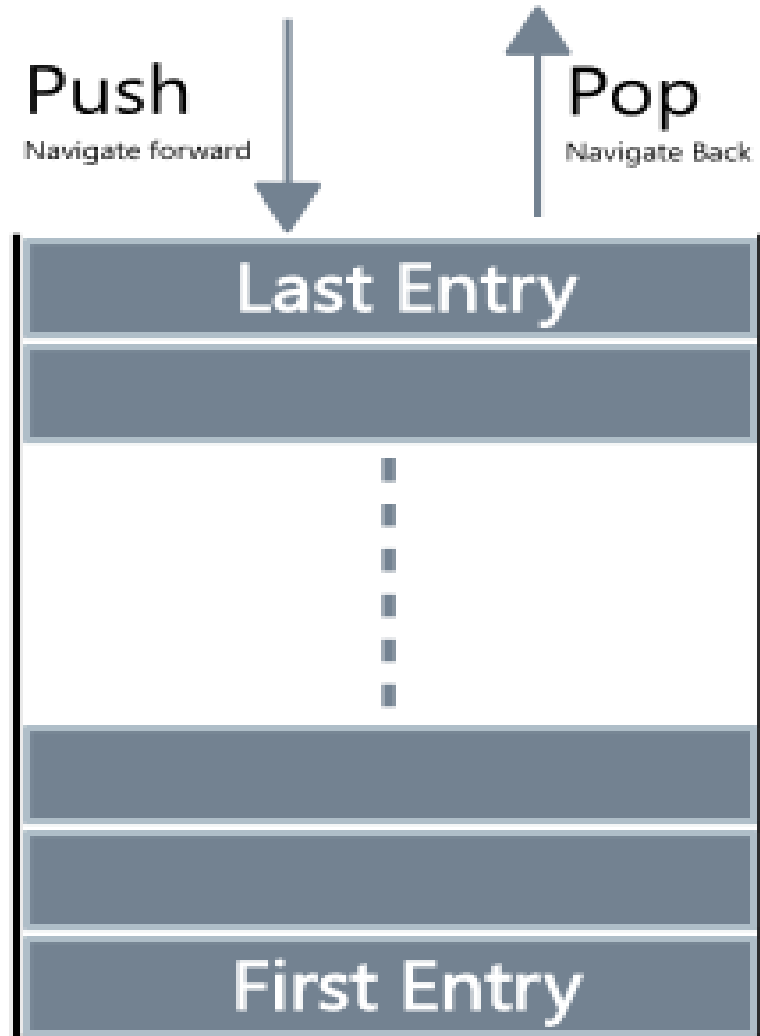


EXAMPLES OF STACK:



Operations that can be performed on STACK:

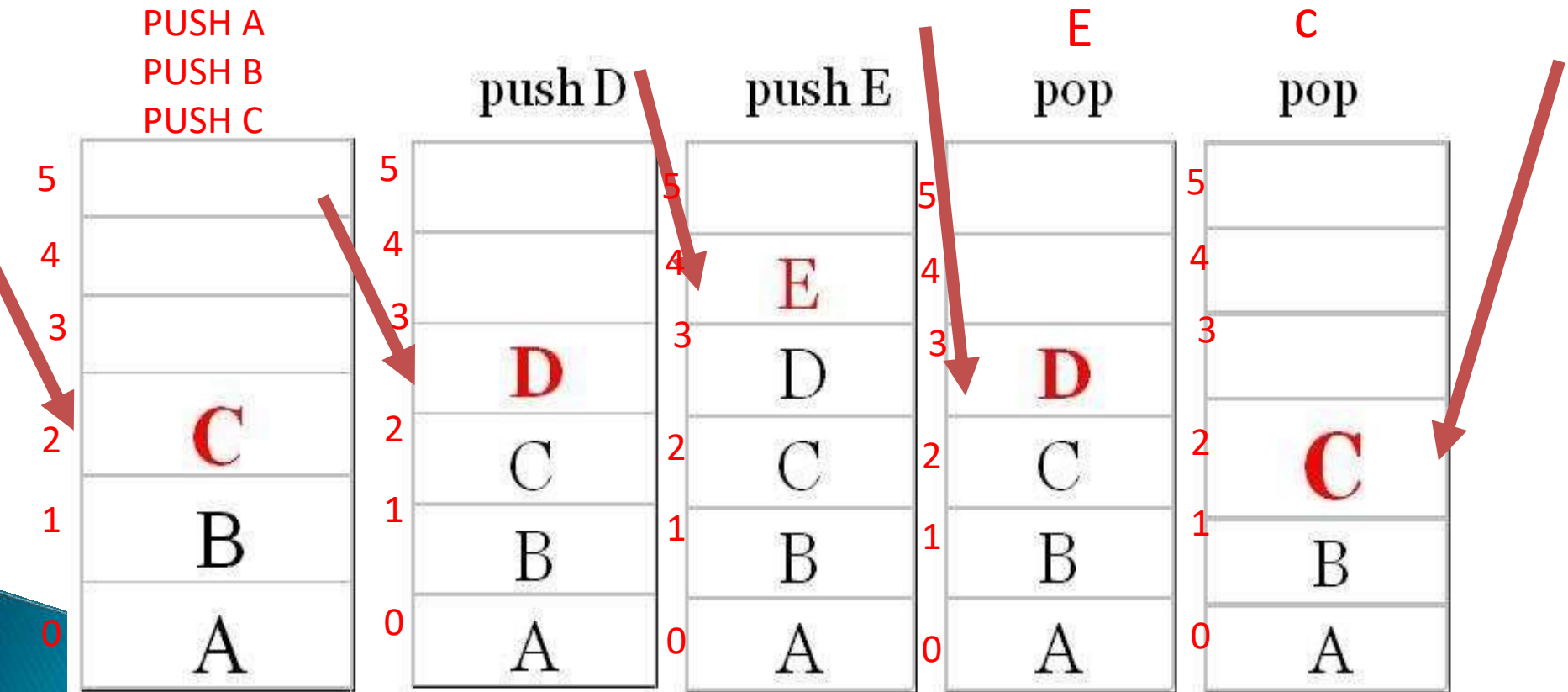
- PUSH.
- POP.



PUSH : It is used to insert items into the stack.

POP: It is used to delete items from stack.

TOP: It represents the current location of data in stack.



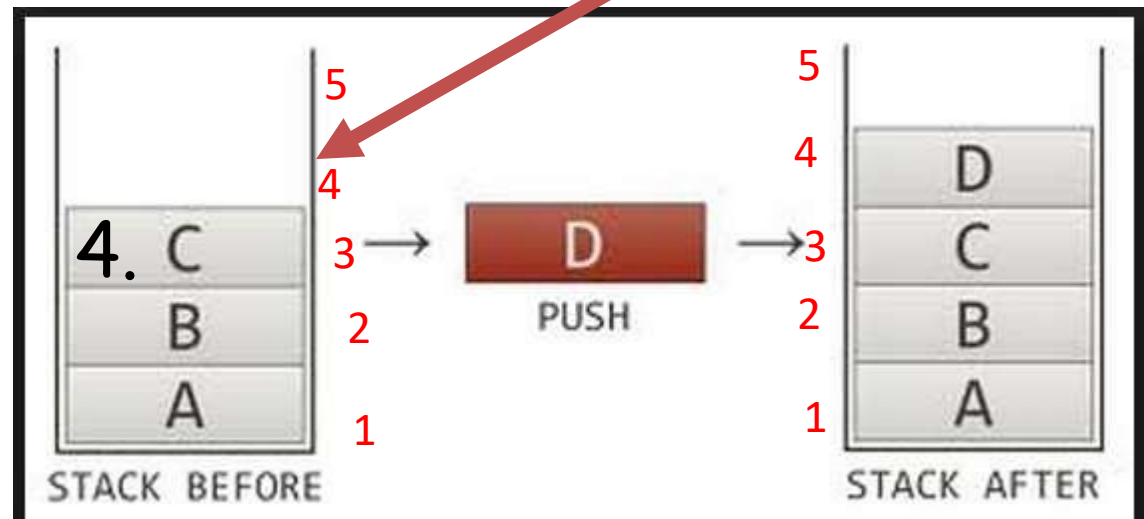
ALGORITHM OF INSERTION IN

STACK: (PUSH) **top** : pointer to stack, 0 Init

max: size of stack

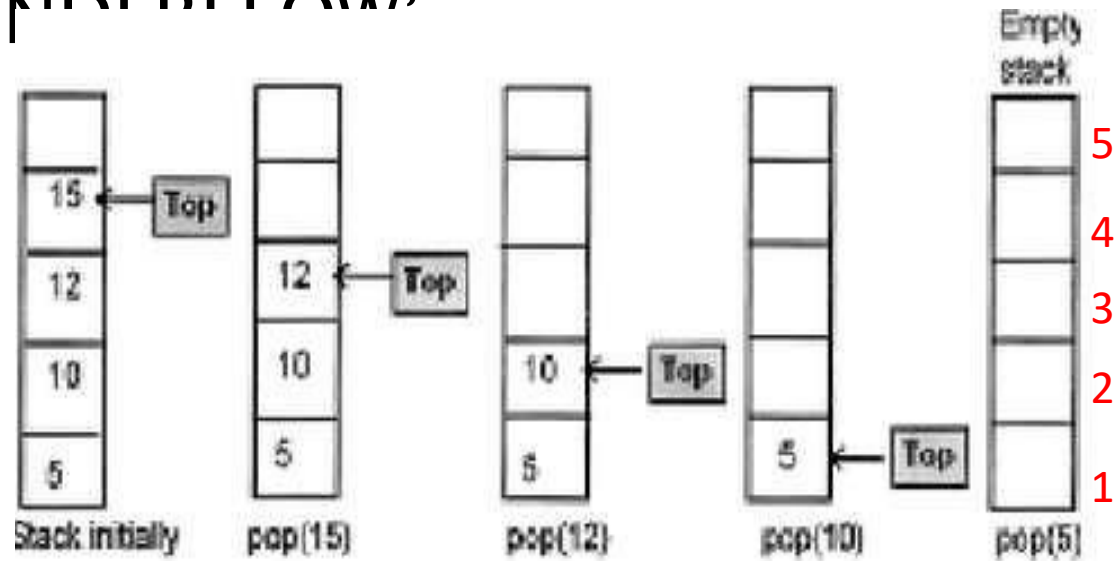
stack[size]: stack as array
declaration

1. Push(item)
2. {
3. If top=max then
 print 'STACK OVERFLOW'
 exit
 else
 3. top=top+1
 stack[top]=item
 End if
}



ALGORITHM OF DELETION IN STACK: (POP)

1. Pop ()
2. {
3. If top=0 then
 print 'STACK UNDERFLOW'
 exit
 else
3. item=stack[top]
 top=top-1
4. End if
5. }



ALGORITHM OF DISPLAY IN

STACK: Display()

{

 If top=0 then

 print 'STACK EMPTY'

 exit

 else

 for i=top to 0

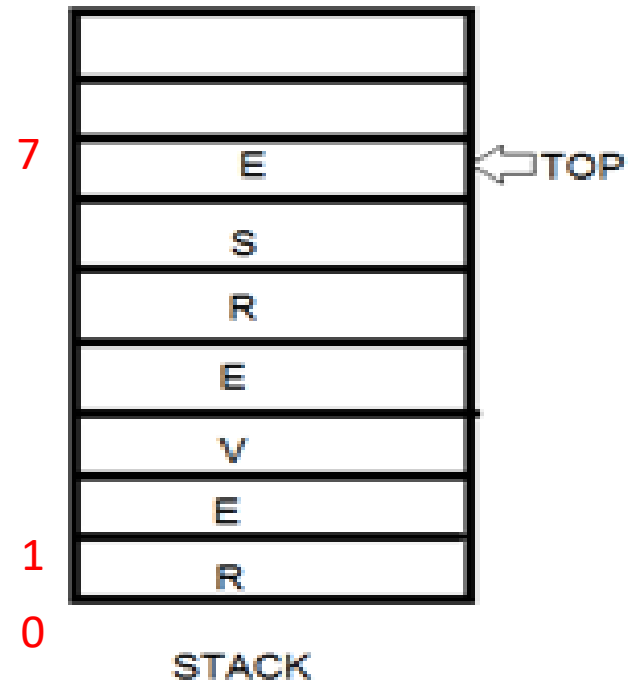
 print stack[i]

 i=i-1

 end for

 end if

}



APPLICATIONS OF STACKS ARE:

I. Reversing Strings:

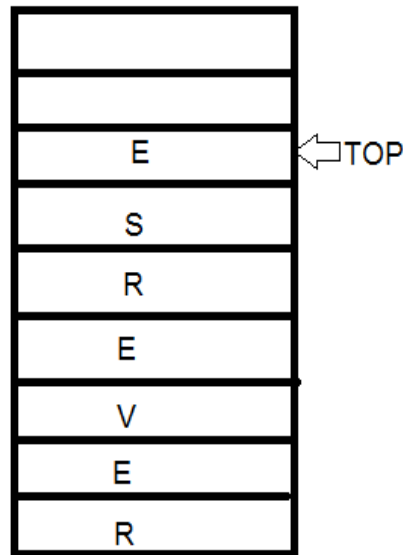
- A simple application of stack is reversing strings. To reverse a string, the characters of string are pushed onto the stack one by one as the string is read from left to right.
 - Once all the characters of string are pushed onto stack, they are popped one by one. Since the character last pushed in comes out first, subsequent pop operation results in the reversal of the string.
- 

For example:

To reverse the string 'REVERSE' the string is read from left to right and its characters are pushed . LIKE:

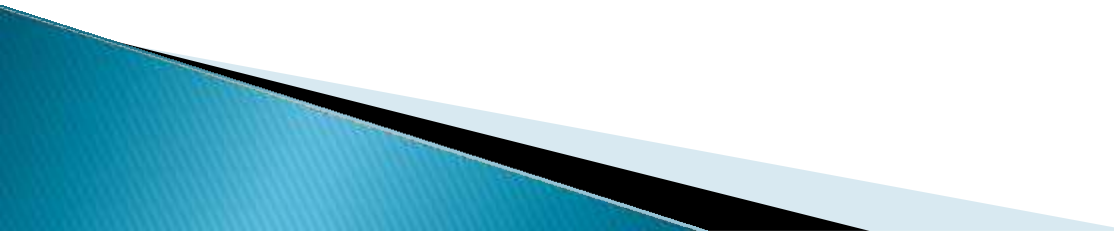
STRING IS:

REVERSE



STACK

II. Checking the validity of an expression containing nested parenthesis:

- Stacks are also used to check whether a given arithmetic expressions containing nested parenthesis is properly parenthesized.
 - The program for checking the validity of an expression verifies that for each left parenthesis braces or bracket ,there is a corresponding closing symbol and symbols are appropriately nested.
- 

For example:

VALID INPUTS	INVALID INPUTS
{ }	{ (}
({ [] })	([(()])
{ [] () }	{ } [])
[{ ({ } [] ({ }) }]	[{) } ([] }]

III. Evaluating arithmetic expressions:

INFIX notation:

The general way of writing arithmetic expressions is known as infix notation. e.g, (a+b)
operand1 operator operand2

PREFIX notation: e.g, +AB
operator operand1 operand2

()
* / %
+ -

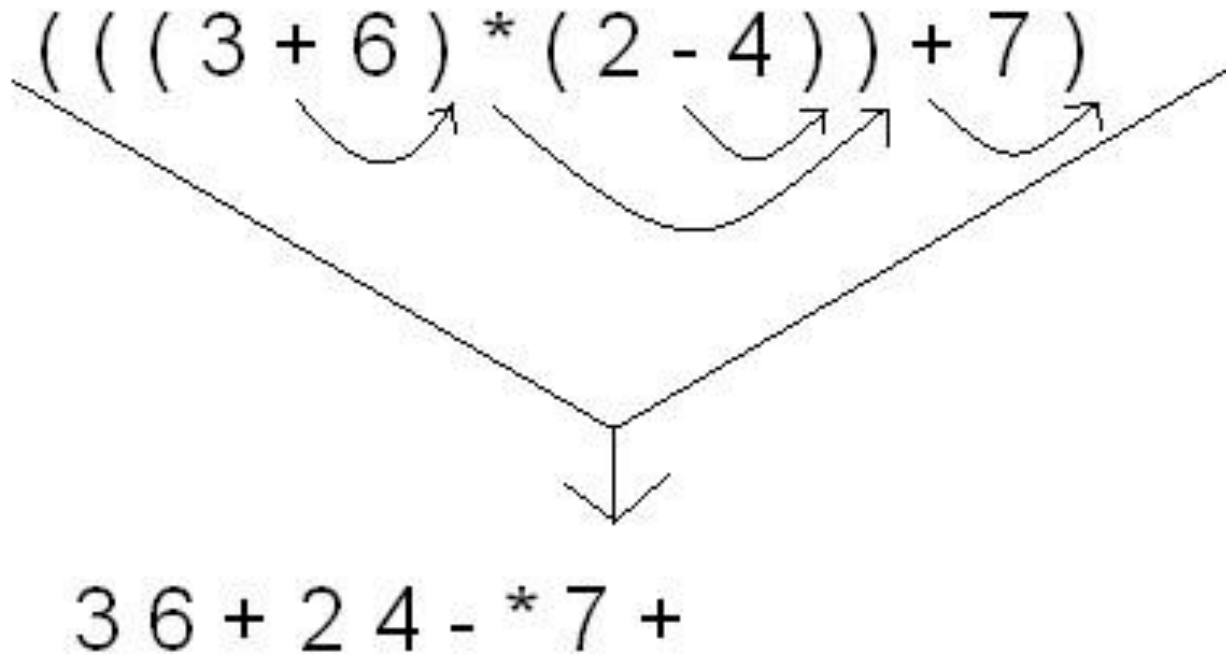
POSTFIX notation: e.g: AB+
operand1 operand2 operator

infix notation: $a+(b*c)$

prefix : $a + *bc \Rightarrow + a*bc$

postfix: $a + bc* \Rightarrow abc*+$

Infix to postfix conversion



examples

Infix

$2+3*4$

$a*b+5$

$(1+2)*7$

$a*b/c$

$(a/(b-c+d))*(e-a)*c$

$a/b-c+d*e-a*c$

$(a/(b-c+d))*(e-a)*c$

$(a/(b-c+d))*ea-*c$

$(a/bc-d+)*ea-*c$

$a\ bc-d+ / * ea-*c$

$a\ bc-d+ / ea- / * c$

$a\ bc-d+ / ea- / c *$

$(A / b) - c + (d * e) - (a * c)$

1 $(ab/) - c + (de*) - (ac*)$

2 $(ab/c-) + (de*) - (ac*)$

3 $(ab/c- de* +) - (ac*)$

4 $ab/c-de*+ ac* -$

examples

Infix	Postfix
$2+3*4$	$234*+$
$a*b+5$	$ab*5+$
$(1+2)*7$	$12+7*$
$a*b/c$	$ab*c/$
$(a/(b-c+d))*(e-a)*c$	$abc-d+ / ea-*c*$
$a/b-c+d*e-a*c$	$ab/c-de*ac*-$

Let, **X** is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression **Y**.

1. Print operands as they arrive.
2. If the stack is empty or contains a left parenthesis on top, push the Incoming operator onto the stack.
3. If the Incoming symbol is '(', push it onto stack.
4. If the Incoming symbol is ')', pop the stack & add operators to output Y until left parenthesis is found.
5. If incoming symbol has higher precedence than the top of the stack, push it onto stack.
6. If the Incoming symbol has lower precedence than the top of the stack, then pop and add to the output. Then test the incoming operator against the new top of the stack.
7. If the incoming operator has equal precedence with the top of the stack, use associativity rule.
8. At the end of the expression pop all the operators of stack.

Associativity **L to R** then pop & print the top of the stack & then push in the incoming operator.

R to L then push the incoming operator

Example: $A - B / C \times D + E$

Example: $A - B / C \times D + E$

Post fix: $ABC/DX-E+$

Example

Suppose we want to convert $2*3/(2-1)+5*3$ into Postfix form,

Expression	Stack	Output
2	Empty	2
*	*	2
3	*	23
/	/	23*
(	/(	23*
2	/(	23*2
-	/(-	23*2
1	/(-	23*21
)	/	23*21-
+	+	23*21-/
5	+	23*21-/5
*	++	23*21-/53
3	++	23*21-/53
	Empty	23*21-/53*+

$a*b/(c-d)+e*f$

So, the Postfix Expression is $23*21-/53*+$

Conversion of INFIX to POSTFIX conversion:

Example: $2+(4-1)*3$

$2+41-*3$

$2+41-3*$

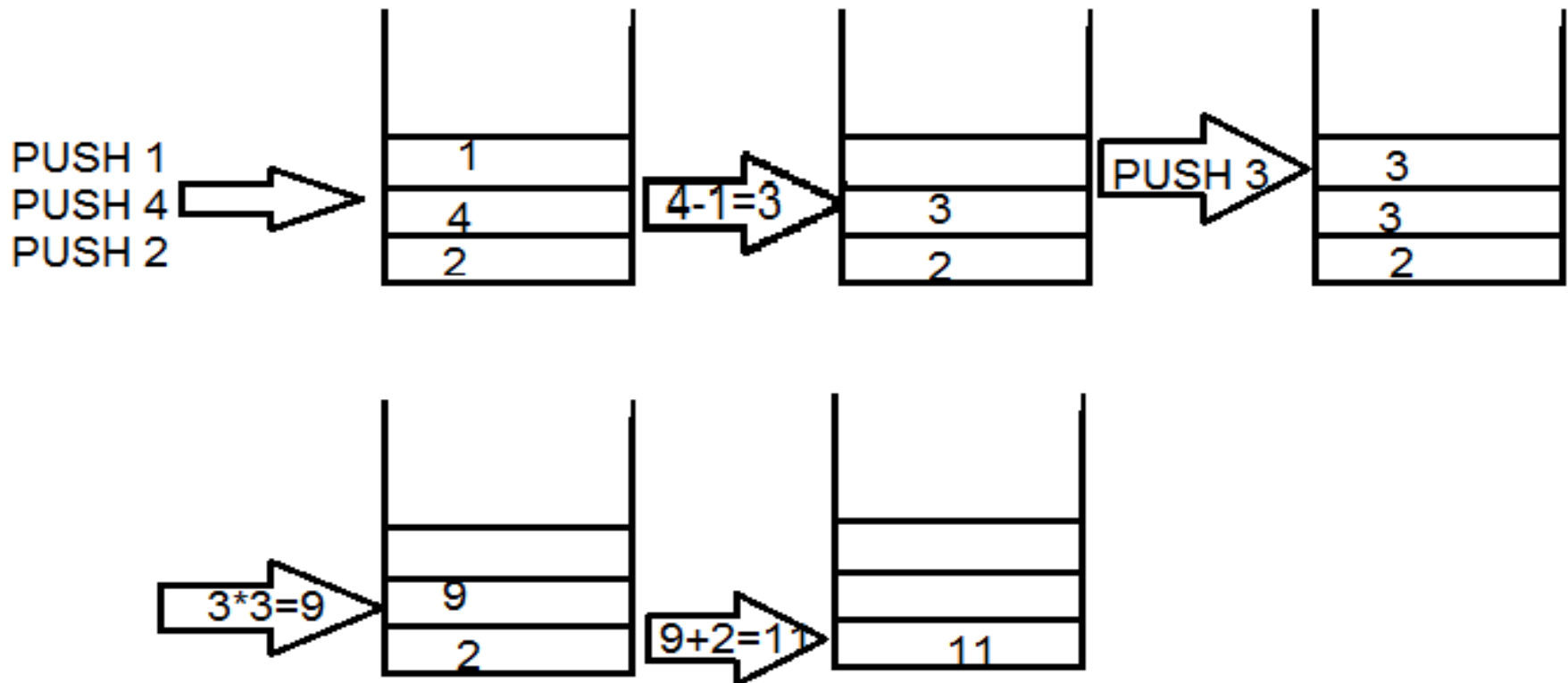
$241-3*+$

step1

step2

step3

step4



CONVERSION OF INFIX INTO POSTFIX

2+(4-1)*3) into 241-3*+

CURRENT SYMBOL	ACTION PERFORMED	STACK STATUS	POSTFIX EXPRESSION
(	PUSH C	C	2
2			2
+	PUSH +	(+	2
(	PUSH (	(+(	24
4			24
-	PUSH -	(+(-	241
1	POP		241-
)		(+	241-
*	PUSH *	(+*	241-
3			241-3
	POP *		241-3*
	POP +		241-3*+
)			

Postfix evaluation

Begin

for each character ch in the postfix expression,
do

if ch is an operator $\odot$, then

a := pop first element from stack

b := pop second element from the stack

res := b $\odot$ a

push res into the stack

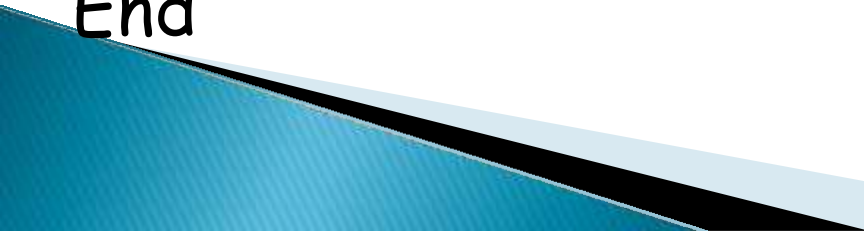
else if ch is an operand, then

add ch into the stack

done

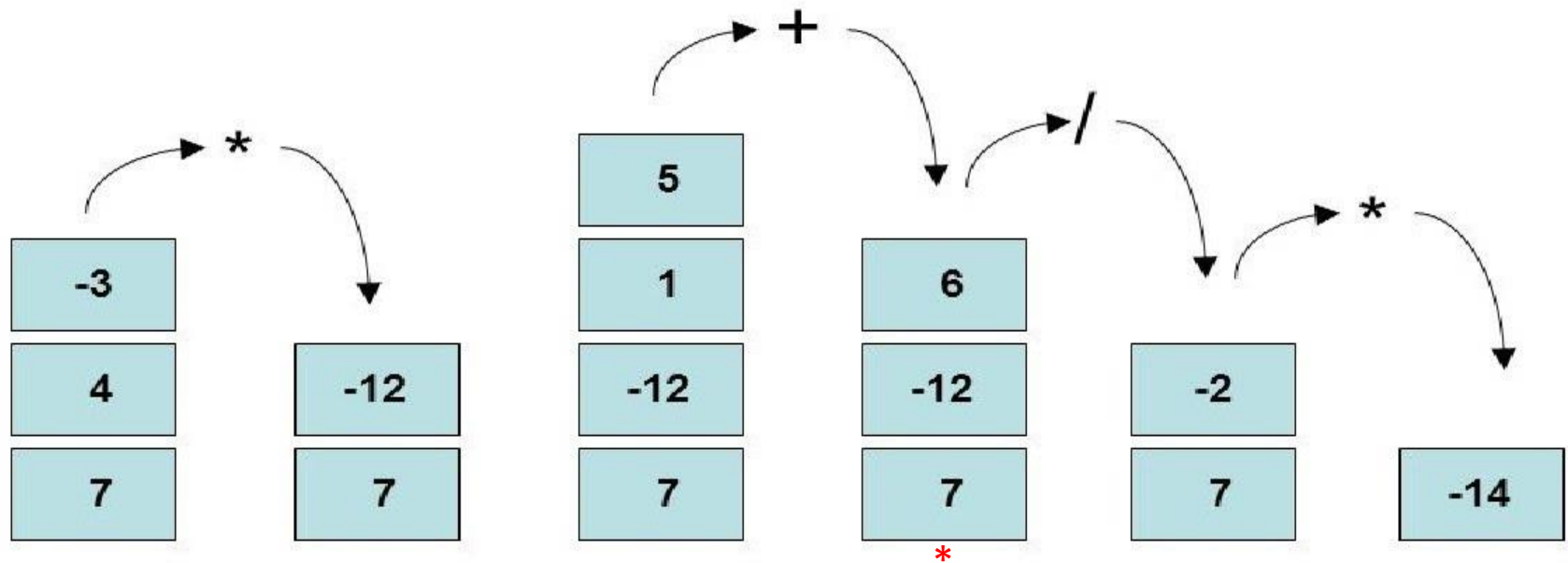
return element of stack top

End



Evaluating Postfix Expressions

- Expression = 7 4 -3 * 1 5 + / *



Ch=~~7~~, ~~4~~, ~~-3~~, ~~*~~,
~~1~~, ~~5~~, ~~+~~, ~~/~~, ~~*~~

A= -2
B= 7
Res= -14